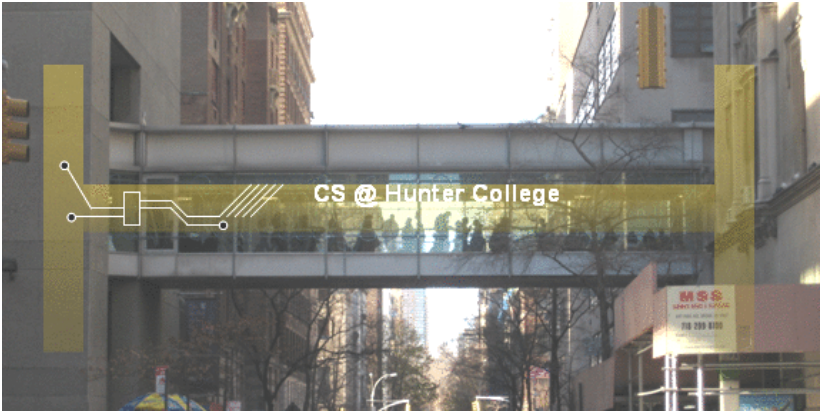
CSci 127: Introduction to Computer Science



hunter.cuny.edu/csci

Outline

- Syllabus & Course Overview
- Introduction to Python
- Turtles & Definite Loops (`for`-loops)
- Algorithms
- Wrap Up

Outline

- Syllabus & Course Overview
- Introduction to Python
- Turtles & Definite Loops (`for`-loops)
- Algorithms
- Wrap Up

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.
- **Prepare & Participate in Lecture:**

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.
- **Prepare & Participate in Lecture:** Hands-on technical challenges every lecture. Get more out of lecture by reviewing your notes and labs.

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.
- **Prepare & Participate in Lecture:** Hands-on technical challenges every lecture. Get more out of lecture by reviewing your notes and labs.
- **Respect & Integrity:** Treat all with respect and do not submit others' work as your own.

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.
- **Prepare & Participate in Lecture:** Hands-on technical challenges every lecture. Get more out of lecture by reviewing your notes and labs.
- **Respect & Integrity:** Treat all with respect and do not submit others' work as your own.
- **End Products:** Enhanced analytic reasoning skills; Programming experience in Python, HTML, and C++.

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.
- **Prepare & Participate in Lecture:** Hands-on technical challenges every lecture. Get more out of lecture by reviewing your notes and labs.
- **Respect & Integrity:** Treat all with respect and do not submit others' work as your own.
- **End Products:** Enhanced analytic reasoning skills; Programming experience in Python, HTML, and C++.
- **Emergencies:** *You may have to miss a class for an emergency, family, or a personal reason.*

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.
- **Prepare & Participate in Lecture:** Hands-on technical challenges every lecture. Get more out of lecture by reviewing your notes and labs.
- **Respect & Integrity:** Treat all with respect and do not submit others' work as your own.
- **End Products:** Enhanced analytic reasoning skills; Programming experience in Python, HTML, and C++.
- **Emergencies:** *You may have to miss a class for an emergency, family, or a personal reason.*
  For your privacy, no explanation is needed.

Welcome: Expectations

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **Background Material:** No previous programming knowledge assumed.
- **Prepare & Participate in Lecture:** Hands-on technical challenges every lecture. Get more out of lecture by reviewing your notes and labs.
- **Respect & Integrity:** Treat all with respect and do not submit others' work as your own.
- **End Products:** Enhanced analytic reasoning skills; Programming experience in Python, HTML, and C++.
- **Emergencies:** *You may have to miss a class for an emergency, family, or a personal reason.*
  For your privacy, no explanation is needed.
  We will automatically drop and/or replace lowest grades (see syllabus). If you don't miss any, we will still drop and replace lowest grades.

Class Policies: Resources

Webpage & Syllabus:
`https://huntercsci127.github.io/summer26.html`

- **Brightspace:** Automatically generated from CUNYFirst within 24 hours after enrollment.

Class Policies: Resources

Webpage & Syllabus:
`https://huntercsci127.github.io/summer26.html`

- **Brightspace:** Automatically generated from CUNYFirst within 24 hours after enrollment.
- **Gradescope:** Invitations will be sent out today.

Class Policies: Resources

Webpage & Syllabus:
`https://huntercsci127.github.io/summer26.html`

- **Brightspace:** Automatically generated from CUNYFirst within 24 hours after enrollment.
- **Gradescope:** Invitations will be sent out today.
- **Python Set-up:** Using Python 3.10+;
  See Lab 1 for installation guides.

Class Policies: Resources

Webpage & Syllabus:
`https://huntercsci127.github.io/summer26.html`

- **Brightspace:** Automatically generated from CUNYFirst within 24 hours after enrollment.
- **Gradescope:** Invitations will be sent out today.
- **Python Set-up:** Using Python 3.10+;
  See Lab 1 for installation guides.
- **Textbook:** Free, on-line book:
  How to Think Like a Computer Scientist.

Class Policies: Grading

Webpage & Syllabus:
https://huntercsci127.github.io/summer26.html

- **Participation/Lecture Slips** (10%): Design & coding challenges in lecture.

Class Policies: Grading

Webpage & Syllabus:
https://huntercsci127.github.io/summer26.html

- **Participation/Lecture Slips** (10%): Design & coding challenges in lecture.
- **Homework** (30%): Reinforce concepts.

Class Policies: Grading

Webpage & Syllabus:
https://huntercsci127.github.io/summer26.html

- **Participation/Lecture Slips** (10%): Design & coding challenges in lecture.
- **Homework** (30%): Reinforce concepts.
- **Quizzes** (30%): Weekly written quizzes at the beginning of class.

Class Policies: Grading

Webpage & Syllabus:
https://huntercsci127.github.io/summer26.html

- **Participation/Lecture Slips** (10%): Design & coding challenges in lecture.
- **Homework** (30%): Reinforce concepts.
- **Quizzes** (30%): Weekly written quizzes at the beginning of class.
- **Final Exam:**(30%): July 7th, 12-2pm.

Class Policies: Grading

Webpage & Syllabus:
https://huntercsci127.github.io/summer26.html

- **Participation/Lecture Slips** (10%): Design & coding challenges in lecture.
- **Homework** (30%): Reinforce concepts.
- **Quizzes** (30%): Weekly written quizzes at the beginning of class.
- **Final Exam:**(30%): July 7th, 12-2pm.

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **General Rule:** All typing should be your own.

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **General Rule:** All typing should be your own.
  If you look up answer (e.g. friend/chatGPT), do not submit that work as your own.

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **General Rule:** All typing should be your own.
  If you look up answer (e.g. friend/chatGPT), do not submit that work as your own.
- **College Policy:** *Hunter College regards acts of academic dishonesty (e.g., plagiarism, cheating on examinations, obtaining unfair advantage, and falsification of records & official documents) as serious offenses against the values of intellectual honesty. The College is committed to enforcing the CUNY Policy on Academic Integrity.*

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **General Rule:** All typing should be your own.
  If you look up answer (e.g. friend/chatGPT), do not submit that work as your own.
- **College Policy:** *Hunter College regards acts of academic dishonesty (e.g., plagiarism, cheating on examinations, obtaining unfair advantage, and falsification of records & official documents) as serious offenses against the values of intellectual honesty. The College is committed to enforcing the CUNY Policy on Academic Integrity.*
- **Reporting:** All incidents reported to Student Conduct, VP Student Affairs, & Computer Science Department.

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **General Rule:** All typing should be your own.
  If you look up answer (e.g. friend/chatGPT), do not submit that work as your own.
- **College Policy:** *Hunter College regards acts of academic dishonesty (e.g., plagiarism, cheating on examinations, obtaining unfair advantage, and falsification of records & official documents) as serious offenses against the values of intellectual honesty. The College is committed to enforcing the CUNY Policy on Academic Integrity.*
- **Reporting:** All incidents reported to Student Conduct, VP Student Affairs, & Computer Science Department.
- **Common Violations:** Copied quiz answers, phone in quiz room, or MOSS scores $> 90\%$ on homework.

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **General Rule:** All typing should be your own.
  If you look up answer (e.g. friend/chatGPT), do not submit that work as your own.
- **College Policy:** *Hunter College regards acts of academic dishonesty (e.g., plagiarism, cheating on examinations, obtaining unfair advantage, and falsification of records & official documents) as serious offenses against the values of intellectual honesty. The College is committed to enforcing the CUNY Policy on Academic Integrity.*
- **Reporting:** All incidents reported to Student Conduct, VP Student Affairs, & Computer Science Department.
- **Common Violations:** Copied quiz answers, phone in quiz room, or MOSS scores $> 90\%$ on homework.
- **Standard Sanctions:** (Recommended)
  1^{st}: 0 for assignment & meet with student conduct.

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

- **General Rule:** All typing should be your own.
  If you look up answer (e.g. friend/chatGPT), do not submit that work as your own.
- **College Policy:** *Hunter College regards acts of academic dishonesty (e.g., plagiarism, cheating on examinations, obtaining unfair advantage, and falsification of records & official documents) as serious offenses against the values of intellectual honesty. The College is committed to enforcing the CUNY Policy on Academic Integrity.*
- **Reporting:** All incidents reported to Student Conduct, VP Student Affairs, & Computer Science Department.
- **Common Violations:** Copied quiz answers, phone in quiz room, or MOSS scores $> 90\%$ on homework.
- **Standard Sanctions:** (Recommended)
  1^{st}: 0 for assignment & meet with student conduct.
  2^{nd}: F for course grade & temp. transcript note.

Class Policies: Academic Integrity

Webpage & Syllabus:

https://huntercsci127.github.io/summer26.html

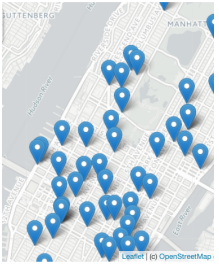
- **General Rule:** All typing should be your own.
  If you look up answer (e.g. friend/chatGPT), do not submit that work as your own.
- **College Policy:** *Hunter College regards acts of academic dishonesty (e.g., plagiarism, cheating on examinations, obtaining unfair advantage, and falsification of records & official documents) as serious offenses against the values of intellectual honesty. The College is committed to enforcing the CUNY Policy on Academic Integrity.*
- **Reporting:** All incidents reported to Student Conduct, VP Student Affairs, & Computer Science Department.
- **Common Violations:** Copied quiz answers, phone in quiz room, or MOSS scores $> 90\%$ on homework.
- **Standard Sanctions:** (Recommended)
  1^{st}: 0 for assignment & meet with student conduct.
  2^{nd}: F for course grade & temp. transcript note.

Course Description

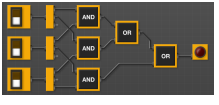
Webpage & Syllabus:
https://huntercsci127.github.io/summer26.html

CSci 127: Introduction to Computer Science
Catalog Description: 3 hours, 3 credits: This course presents an overview of computer science (CS) with an emphasis on ***problem-solving and computational thinking through 'coding'****: computer programming for beginners.*
This course is pre-requisite to several introductory core courses in the CS Major. The course is also required for the CS minor. MATH 12500 or higher is strongly recommended as a co-req for intended Majors.

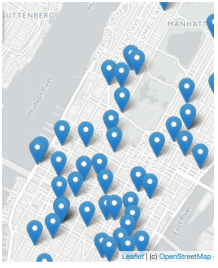
Syllabus: Topics

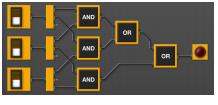


- **This course assumes no previous programming experience.**



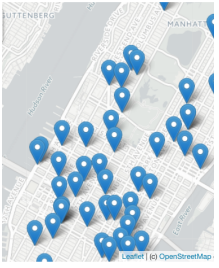
Syllabus: Topics

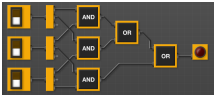




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:

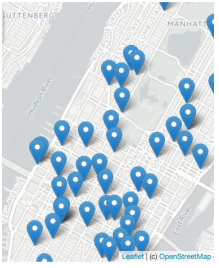
Syllabus: Topics

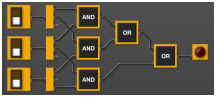




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,

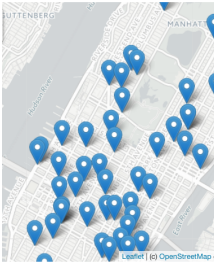
Syllabus: Topics

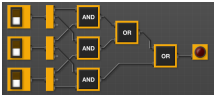




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,
  - Apply those ideas to different problems (e.g. analyzing & mapping data),

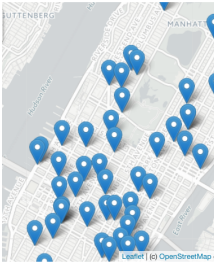
Syllabus: Topics

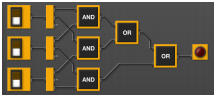




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,
  - Apply those ideas to different problems (e.g. analyzing & mapping data),
  - See constructs again:

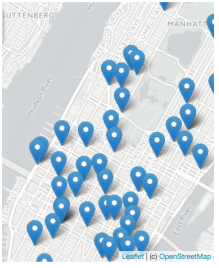
Syllabus: Topics

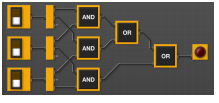




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,
  - Apply those ideas to different problems (e.g. analyzing & mapping data),
  - See constructs again:
    - for logical circuits,

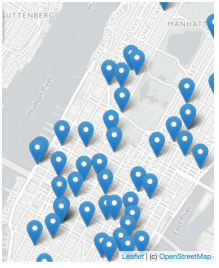
Syllabus: Topics

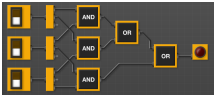




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,
  - Apply those ideas to different problems (e.g. analyzing & mapping data),
  - See constructs again:
    - for logical circuits,
    - for Unix command line interface,

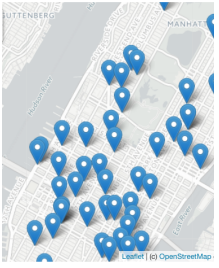
Syllabus: Topics

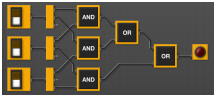




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,
  - Apply those ideas to different problems (e.g. analyzing & mapping data),
  - See constructs again:
    - for logical circuits,
    - for Unix command line interface,
    - for the markup language for github,

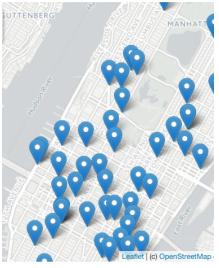
Syllabus: Topics

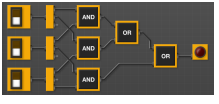




- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,
  - Apply those ideas to different problems (e.g. analyzing & mapping data),
  - See constructs again:
    - for logical circuits,
    - for Unix command line interface,
    - for the markup language for github,
    - for simplified machine languages, &

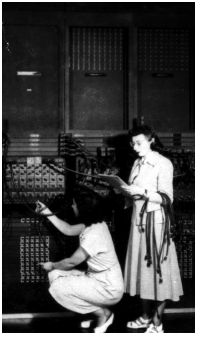
Syllabus: Topics





- **This course assumes no previous programming experience.**
- Organized like a fugue, with variations on this theme:
  - Introduce coding constructs in Python,
  - Apply those ideas to different problems (e.g. analyzing & mapping data),
  - See constructs again:
    - for logical circuits,
    - for Unix command line interface,
    - for the markup language for github,
    - for simplified machine languages, &
    - for C++.

Class Structure



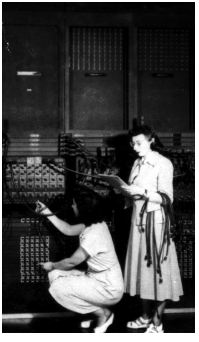
First "computers"

ENIAC, 1945.

Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.

Class Structure



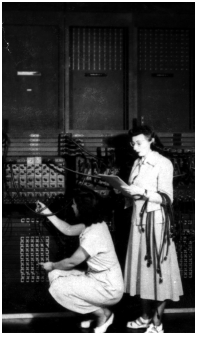
First "computers"

ENIAC, 1945.

Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.

Class Structure



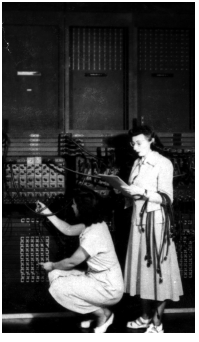
First "computers"

ENIAC, 1945.

Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

Class Structure



First "computers"

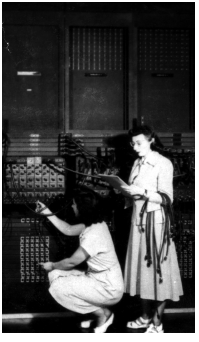
ENIAC, 1945.

Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

Lab Exercises:

Class Structure



First "computers"

ENIAC, 1945.

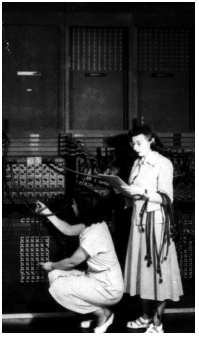
Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

Lab Exercises:

- Can complete at home or at the end of lecture.

Class Structure



First "computers"

ENIAC, 1945.

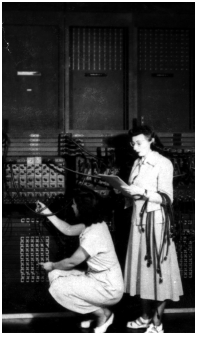
Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

Lab Exercises:

- Can complete at home or at the end of lecture.
- Each lab lists required software tools.

Class Structure



First "computers"

ENIAC, 1945.

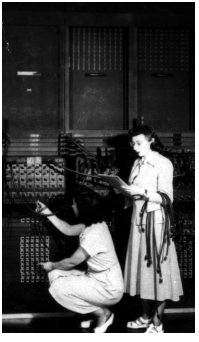
Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

Lab Exercises:

- Can complete at home or at the end of lecture.
- Each lab lists required software tools.
- Basis of programs, quizzes, & final exam.

Class Structure



First "computers"

ENIAC, 1945.

Lecture:

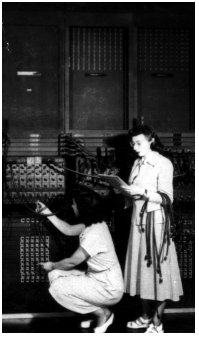
- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

Lab Exercises:

- Can complete at home or at the end of lecture.
- Each lab lists required software tools.
- Basis of programs, quizzes, & final exam.

Quizzes:

Class Structure



First "computers"

ENIAC, 1945.

Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

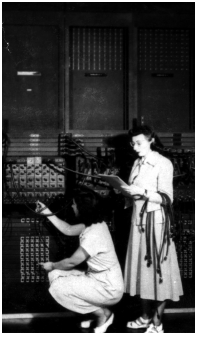
Lab Exercises:

- Can complete at home or at the end of lecture.
- Each lab lists required software tools.
- Basis of programs, quizzes, & final exam.

Quizzes:

- Paper Quiz: Questions follow final exam style.

Class Structure



First "computers"

ENIAC, 1945.

Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

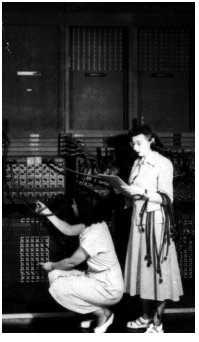
Lab Exercises:

- Can complete at home or at the end of lecture.
- Each lab lists required software tools.
- Basis of programs, quizzes, & final exam.

Quizzes:

- Paper Quiz: Questions follow final exam style.
- For upcoming class days, will happen at the beginning of lecture.

Class Structure



First "computers"

ENIAC, 1945.

Lecture:

- Tues/Thur, 11:40am-2:48pm, HN1001E.
- Mix of explanation and hands on learning.
- Ask questions when you're not sure!!.

Lab Exercises:

- Can complete at home or at the end of lecture.
- Each lab lists required software tools.
- Basis of programs, quizzes, & final exam.

Quizzes:

- Paper Quiz: Questions follow final exam style.
- For upcoming class days, will happen at the beginning of lecture.

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).
- Okay, then could everyone get an A?

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).
- Okay, then could everyone get an A?
  Yes, we are not a filter course. Our goal is for students to succeed in the course and in the CS major and minor.

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).
- Okay, then could everyone get an A?
  Yes, we are not a filter course. Our goal is for students to succeed in the course and in the CS major and minor.
- What happens to my grade if I miss a lecture or quiz?

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).
- Okay, then could everyone get an A?
  Yes, we are not a filter course. Our goal is for students to succeed in the course and in the CS major and minor.
- What happens to my grade if I miss a lecture or quiz?
  We drop missing or low grades on lecture slips, homeworks, quizzes and code reviews.

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).
- Okay, then could everyone get an A?
  Yes, we are not a filter course. Our goal is for students to succeed in the course and in the CS major and minor.
- What happens to my grade if I miss a lecture or quiz?
  We drop missing or low grades on lecture slips, homeworks, quizzes and code reviews.
- Do I have to pass the final to pass the course?

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).
- Okay, then could everyone get an A?
  Yes, we are not a filter course. Our goal is for students to succeed in the course and in the CS major and minor.
- What happens to my grade if I miss a lecture or quiz?
  We drop missing or low grades on lecture slips, homeworks, quizzes and code reviews.
- Do I have to pass the final to pass the course?
  Yes. To demonstrate mastery, you must pass the final exam.

Philosophy (Or Why We Do What We Do)

Grading:

- Do you curve grades?
  No, we grade on your mastery of the material and do not have a set number of A's, B's, C's that we curve grades to match (i.e. your demonstrated mastery over your relative performance to the class).
- Okay, then could everyone get an A?
  Yes, we are not a filter course. Our goal is for students to succeed in the course and in the CS major and minor.
- What happens to my grade if I miss a lecture or quiz?
  We drop missing or low grades on lecture slips, homeworks, quizzes and code reviews.
- Do I have to pass the final to pass the course?
  Yes. To demonstrate mastery, you must pass the final exam.
  We will end most lectures with past final exam questions and review.

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.
  Actively using knowledge increases your brain's ability to retain knowledge.

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.
  Actively using knowledge increases your brain's ability to retain knowledge.
- I like working by myself. Why do I have to work in groups during class?

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.
  Actively using knowledge increases your brain's ability to retain knowledge.
- I like working by myself. Why do I have to work in groups during class?
  Active learning increases student performance.

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.
  Actively using knowledge increases your brain's ability to retain knowledge.
- I like working by myself. Why do I have to work in groups during class?
  Active learning increases student performance.
  Also, it's excellent practice explaining technical ideas (i.e. tech interviews).
- What's the best way to study for this course?

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.
  Actively using knowledge increases your brain's ability to retain knowledge.
- I like working by myself. Why do I have to work in groups during class?
  Active learning increases student performance.
  Also, it's excellent practice explaining technical ideas (i.e. tech interviews).
- What's the best way to study for this course?
  - *Most efficient way: do the programs*

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.
  Actively using knowledge increases your brain's ability to retain knowledge.
- I like working by myself. Why do I have to work in groups during class?
  Active learning increases student performance.
  Also, it's excellent practice explaining technical ideas (i.e. tech interviews).
- What's the best way to study for this course?
  - *Most efficient way: do the programs (more programs on old finals).*

Philosophy (Or Why We Do What We Do)

Course Structure:

- Why 55 programs assignments? My friend only has to do 10.
  Traditionally, it's 10 long 'all-nighters' assignments.
  While this mimics some jobs, students (particularly those new to programming) master skills better with smaller challenges.
- Why weekly quizzes instead of midterms?
  Weekly quizzes increase pass rates and mastery of material.
  Actively using knowledge increases your brain's ability to retain knowledge.
- I like working by myself. Why do I have to work in groups during class?
  Active learning increases student performance.
  Also, it's excellent practice explaining technical ideas (i.e. tech interviews).
- What's the best way to study for this course?
  - *Most efficient way: do the programs (more programs on old finals).*
  - *Work through the labs.*

Outline

- Syllabus & Course Overview
- Introduction to Python
- Turtles & Definite Loops (`for`-loops)
- Algorithms
- Wrap Up

Programming Languages

- We will write programs: commands to the computer to do something.

Programming Languages

- We will write programs: commands to the computer to do something.
- A **programming language** is a stylized way of writing those commands.

Programming Languages

- We will write programs: commands to the computer to do something.
- A **programming language** is a stylized way of writing those commands.
- Our first language, Python, is popular for its ease-of-use, flexibility, and extendibility, supportive community with hundreds of open source libraries and frameworks.

Programming Languages

- We will write programs: commands to the computer to do something.
- A **programming language** is a stylized way of writing those commands.
- Our first language, Python, is popular for its ease-of-use, flexibility, and extendibility, supportive community with hundreds of open source libraries and frameworks.
- The first lab goes into step-by-step details of getting Python running.

Programming Languages

- We will write programs: commands to the computer to do something.
- A **programming language** is a stylized way of writing those commands.
- Our first language, Python, is popular for its ease-of-use, flexibility, and extendibility, supportive community with hundreds of open source libraries and frameworks.
- The first lab goes into step-by-step details of getting Python running.
- We'll look at the design and basic structure (no worries if you haven't tried it yet).

First Program: `Hello, World!`

Demo in `pythonTutor`

First Program: Hello, World!

```
#Name: Thomas Hunter
#Date: May 28, 2026
#This program prints: Hello, World!

print("Hello, World!")
```

First Program: Hello, World!

```
#Name:  Thomas Hunter
#Date:  September 1, 2017
#This program prints:  Hello, World!
print("Hello, World!")
```

← *Prints*

- Output to the screen is: `Hello, World!`

First Program: `Hello, World!`

```
#Name:  Thomas Hunter
#Date:  September 1, 2017
#This program prints:  Hello, World!
print("Hello, World!")
```

← *Prints*

- Output to the screen is: `Hello, World!`
- We know that `Hello, World!` is a **string** (a sequence of characters) because it is surrounded by quotes

First Program: `Hello, World!`

```
#Name:  Thomas Hunter
#Date:  September 1, 2017
#This program prints:  Hello, World!
print("Hello, World!")
```

← *Prints*

- Output to the screen is: `Hello, World!`
- We know that `Hello, World!` is a **string** (a sequence of characters) because it is surrounded by quotes
- Can replace `Hello, World!` with another string to be printed.

Variations on `Hello, World!`

```
#Name: L-M Miranda
#Date: Hunter College HS `98
#This program prints intro lyrics

print('Get your education,')
```

Spring, 2018, Assembly Hall

Variations on `Hello, World!`

```
#Name: L-M Miranda
#Date: Hunter College HS `98
#This program prints intro lyrics

print('Get your education,')
print("don't forget from whence you came, and")
print("The world's gonna know your name.")
```

- Each print statement writes its output on a new line.
- Results in three lines of output.
- Can use single or double quotes, just need to match.

Outline

- Syllabus & Course Overview
- Introduction to Python
- Turtles & Definite Loops (`for`-loops)
- Algorithms
- Wrap Up

Turtle Graphics

- A simple, whimsical graphics package for Python.

Turtle Graphics

- A simple, whimsical graphics package for Python.
- Dates back to Logo Turtles in the 1960s.

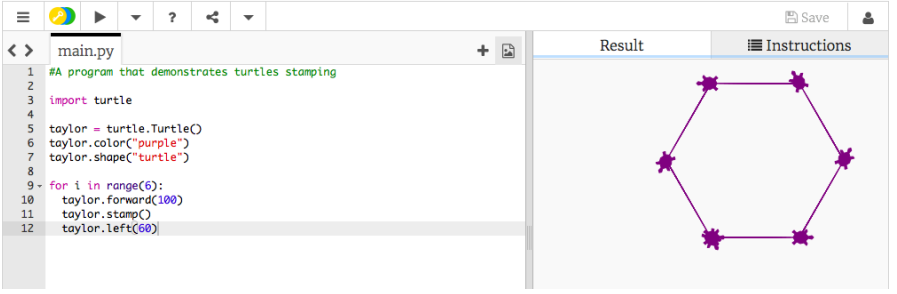
Turtle Graphics

- A simple, whimsical graphics package for Python.
- Dates back to Logo Turtles in the 1960s.
- (Demo from webpage)

Turtle Graphics

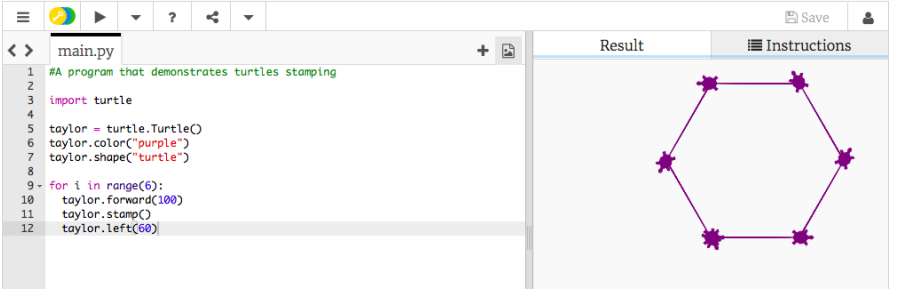
- A simple, whimsical graphics package for Python.
- Dates back to Logo Turtles in the 1960s.
- (Demo from webpage)
- (Fancier turtle demo)

Turtles & Definite Loops



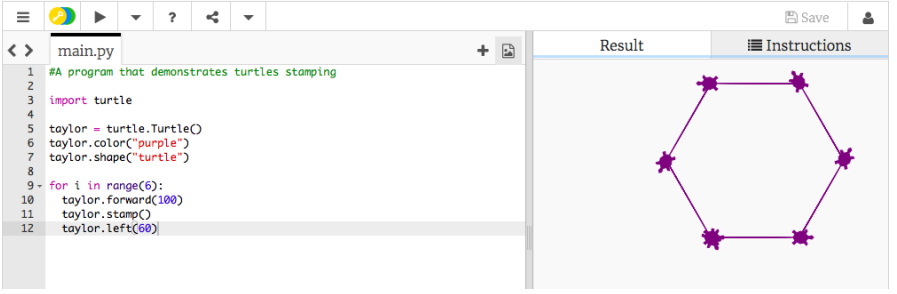
- Creates a turtle **variable**, called `taylor`.

Turtles & Definite Loops



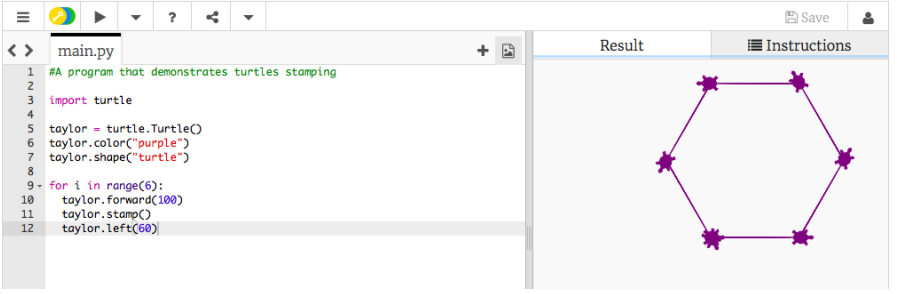
- Creates a turtle **variable**, called `taylor`.
- Changes the color (to purple) and shape (to turtle-shaped).

Turtles & Definite Loops



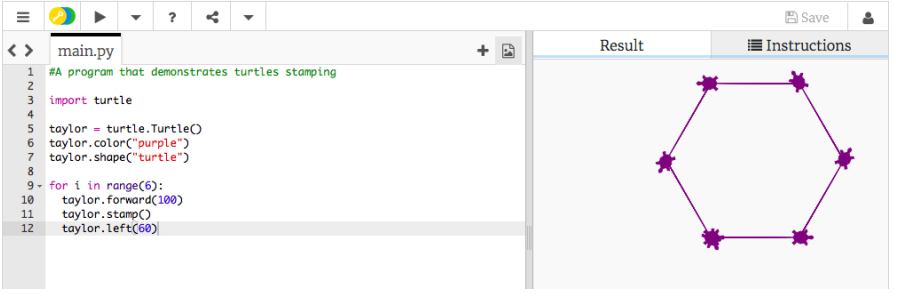
- Creates a turtle **variable**, called `taylor`.
- Changes the color (to purple) and shape (to turtle-shaped).
- Repeats 6 times:

Turtles & Definite Loops



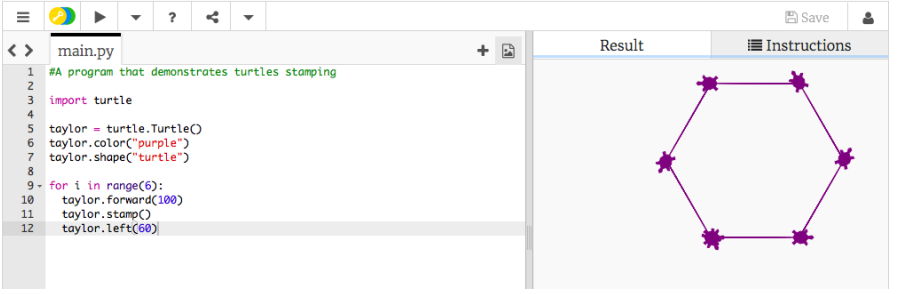
- Creates a turtle **variable**, called `taylor`.
- Changes the color (to purple) and shape (to turtle-shaped).
- Repeats 6 times:
  - Move forward; stamp; and turn left 60 degrees.

Turtles & Definite Loops



- Creates a turtle **variable**, called `taylor`.
- Changes the color (to purple) and shape (to turtle-shaped).
- Repeats 6 times:
  - Move forward; stamp; and turn left 60 degrees.
- Repeats any instructions **indented** in the "loop block"

Turtles & Definite Loops



- Creates a turtle **variable**, called `taylor`.
- Changes the color (to purple) and shape (to turtle-shaped).
- Repeats 6 times:
  - Move forward; stamp; and turn left 60 degrees.
- Repeats any instructions **indented** in the "loop block"
- This is a **definite** loop because it repeats a fixed number of times

Group Work

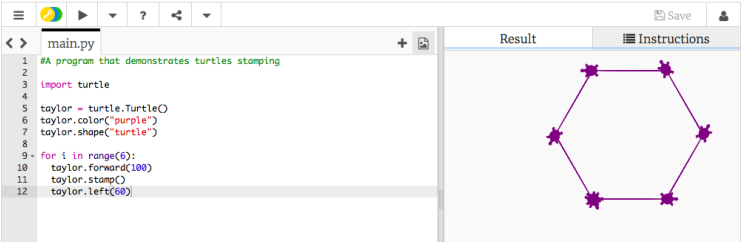
Working in pairs or triples:

1. Write a program that will draw a 10-sided polygon.
2. Write a program that will repeat the line:

   ```
   I'm lookin' for a mind at work!
   ```

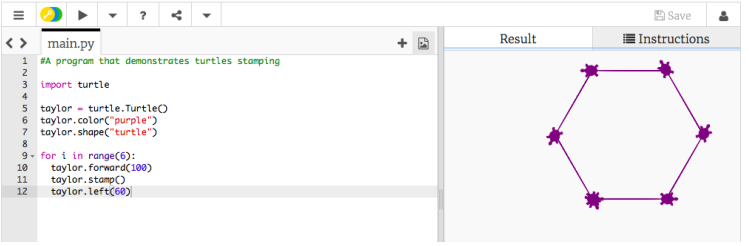
   three times.

Decagon Program



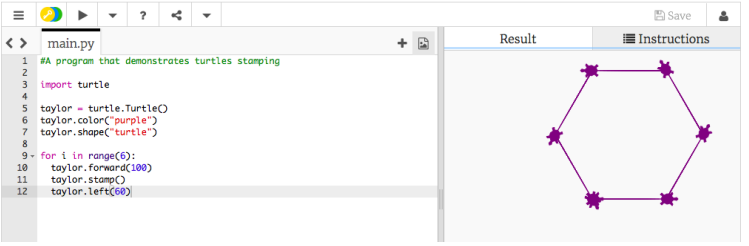
- Start with the hexagon program.

Decagon Program



- Start with the hexagon program.
- Has 10 sides (instead of 6), so change the `range(6)` to `range(10)`.

Decagon Program



- Start with the hexagon program.
- Has 10 sides (instead of 6), so change the `range(6)` to `range(10)`.
- Makes 10 turns (instead of 6),
  so change the `taylor.left(60)` to `taylor.left(360/10)`.

Work Program

2. Write a program that will repeat the line:

   ```
   I'm lookin' for a mind at work!
   ```

   three times.

Work Program

2. Write a program that will repeat the line:

   ```
   I'm lookin' for a mind at work!
   ```

   three times.

- Repeats three times, so, use `range(3)`:

  ```
  for i in range(3):
  ```

Work Program

2. Write a program that will repeat the line:

   ```
   I'm lookin' for a mind at work!
   ```

   three times.

- Repeats three times, so, use `range(3)`:

  ```
  for i in range(3):
  ```

- Instead of turtle commands, repeating a print statement.

Work Program

2. Write a program that will repeat the line:

   ```
   I'm lookin' for a mind at work!
   ```

   three times.

- Repeats three times, so, use `range(3)`:

  ```
  for i in range(3):
  ```

- Instead of turtle commands, repeating a print statement.
- Completed program:

  ```
  # Your name here!
  for i in range(3):
      print("I'm lookin' for a mind at work!")
  ```

Outline

- Syllabus & Course Overview
- Introduction to Python
- Turtles & Definite Loops (`for`-loops)
- Algorithms
- Wrap Up

What is an Algorithm?

From our textbook:

- An algorithm is a process or sequence of steps to be followed to solve a problem.

What is an Algorithm?

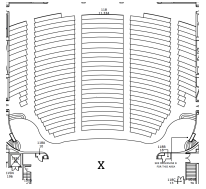
From our textbook:

- An algorithm is a process or sequence of steps to be followed to solve a problem.
- Programming is a skill that allows a computer scientist to take an algorithm and represent it in a notation (a program) that can be executed by a computer.

Outline

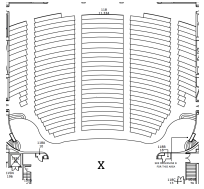
- Syllabus & Course Overview
- Introduction to Python
- Turtles & Definite Loops (`for`-loops)
- Algorithms
- Wrap Up

Recap



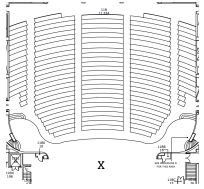
- Writing precise algorithms is difficult.

Recap



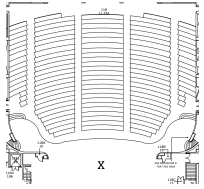
- Writing precise algorithms is difficult.
- In Python, we introduced:

Recap



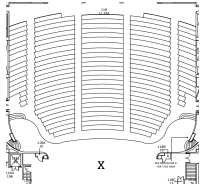
- Writing precise algorithms is difficult.
- In Python, we introduced:
  - strings, or sequences of characters,

Recap



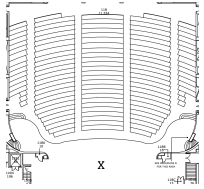
- Writing precise algorithms is difficult.
- In Python, we introduced:
  - strings, or sequences of characters,
  - `print()` statements,

Recap



- Writing precise algorithms is difficult.
- In Python, we introduced:
  - strings, or sequences of characters,
  - `print()` statements,
  - `for`-loops with `range()` statements, &

Recap



- Writing precise algorithms is difficult.
- In Python, we introduced:
  - strings, or sequences of characters,
  - `print()` statements,
  - `for`-loops with `range()` statements, &
  - variables containing turtles.

Remainder of lecture time

- Make sure you have access to the Gradescope before leaving.
- Refer to lab 1 for setting up python and writing first programs.
- Use this time to work on the first 4 assignments (due Friday + Monday).
- (Assignment 5 due date will be pushed back to next Tuesday).
- It is good practice to work on the assignments in advance! Gradescope can and will cause unexpected issues. Please use this time to ask questions when stuck.